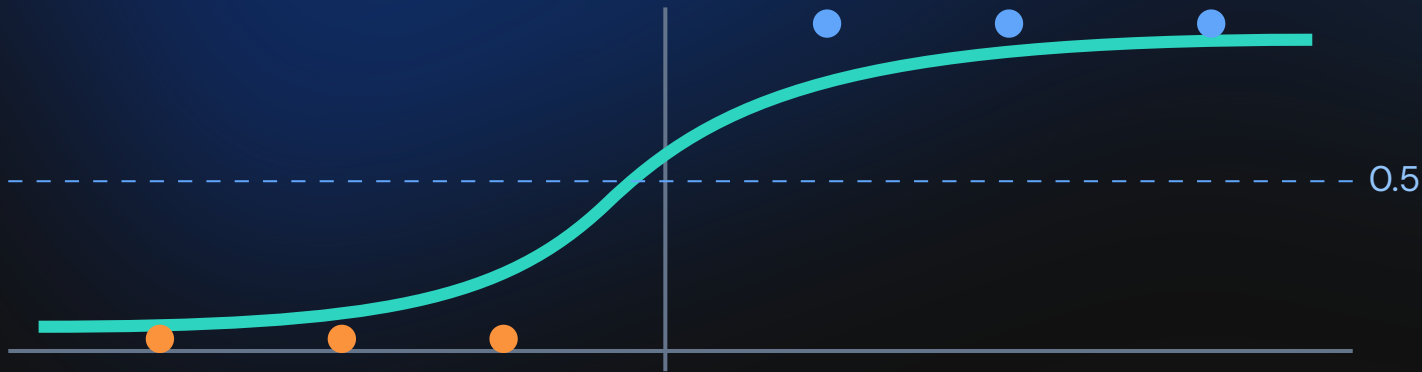


Logistic Regression

From predicting numbers to predicting probabilities

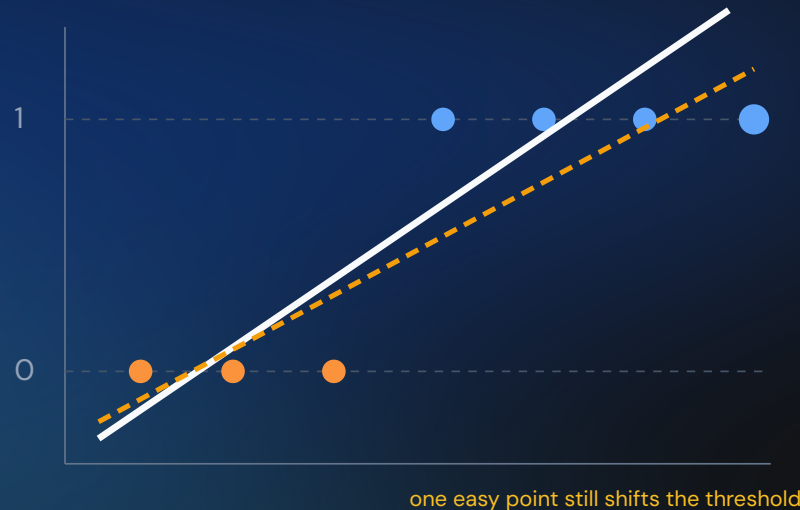
Supervised Learning · Classification



Why Linear Regression Fails Here

- Fit $\hat{y} = x^\top w$ to labels $y \in \{0, 1\}$ and threshold at 0.5?
- Predictions escape $[0, 1]$ — impossible as probabilities
- Far-away, already-correct points still pull the squared-error line
- Classification cares about the boundary, not how far a confident point lies beyond it

We need a bounded probability model **and** a classification loss.



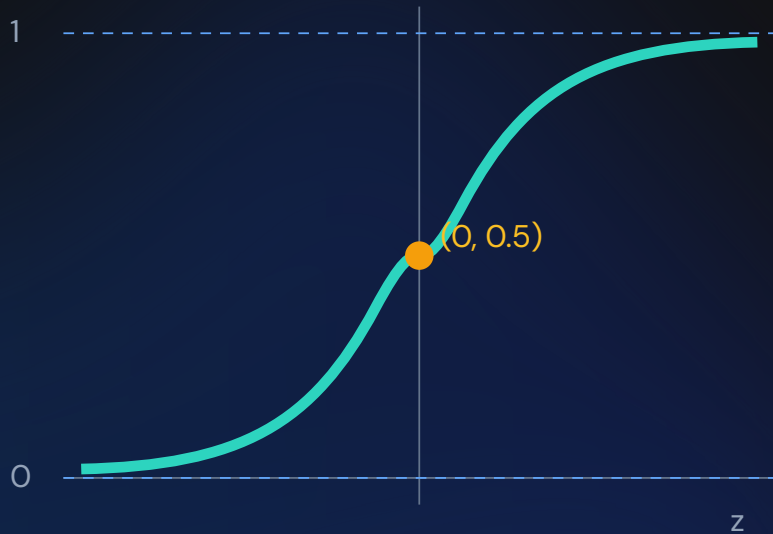
The Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$
$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.linspace(-10, 10, 200)
print(sigmoid(np.array([-10, 0, 10])))
```



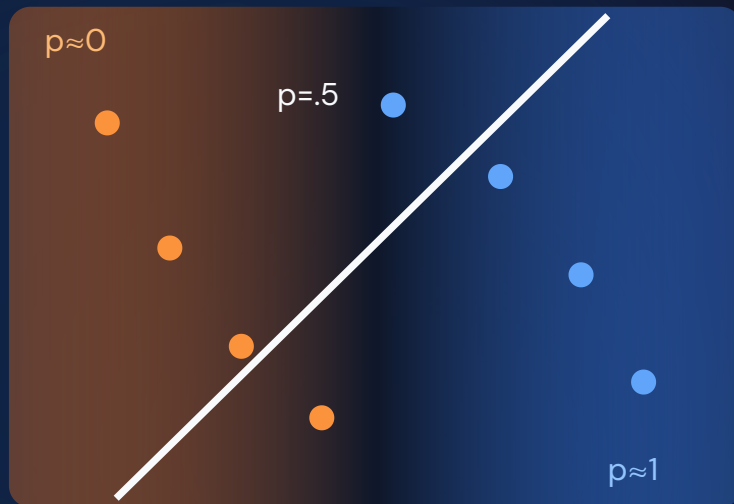
A Linear Boundary, Soft Probabilities

$$\hat{p} = P(y = 1 \mid x) = \sigma(x^\top w)$$

$$\hat{y} = 1 \iff \hat{p} \geq .5 \iff x^\top w \geq 0$$

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
```

```
X, y = make_classification(
    n_samples=200, n_features=2,
    n_redundant=0, random_state=0)
model = LogisticRegression().fit(X, y)
print(model.predict_proba(X[:3]))
print(model.predict(X[:3]))
```



Where Log Loss Comes From: Maximum Likelihood

$$y_i \mid x_i \sim \text{Bernoulli}(\hat{p}_i)$$

$$L(w) = \prod_i \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1-y_i}$$

- Treat each label as one flip of a $\hat{p}_i$ -weighted coin
- Take the log: products become sums, powers become multiplication
- Maximize $\log L(w) \iff$ minimize $-\log L(w) = J(w)$
- $J(w)$ is exactly the log-loss / cross-entropy formula

$$\log L(w) = \sum_i \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

$$J(w) = -\log L(w)$$

Worked examples:

$$y = 1, \quad \hat{p} = 0.8 \quad \Rightarrow \quad -\log(0.8) \approx 0.223$$

$$y = 1, \quad \hat{p} = 0.2 \quad \Rightarrow \quad -\log(0.2) \approx 1.609$$

Log Loss: Chain the Local Derivatives

$$J(w) = - \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

$$w \rightarrow z = x^T w$$

$$\partial z / \partial w = x$$

$$z \rightarrow \hat{p} = \sigma(z)$$

$$\partial \hat{p} / \partial z = \hat{p}(1 - \hat{p})$$

$$\hat{p} \rightarrow J$$

$$-y/\hat{p} + (1-y)/(1-\hat{p})$$

$$\frac{\partial J}{\partial w} = \sum_i (\hat{p}_i - y_i) x_i$$

The Gradient Has a Familiar Shape

$$\nabla_w J(w) = X^\top (\hat{p} - y)$$

Compare linear regression:

$$\nabla_w J_{linear} = X^\top (Xw - y)$$

features $\times$ residual-like error

```
def gradient(X, y, w):
    p = sigmoid(X @ w)
    return X.T @ (p - y)

def loss(X, y, w):
    p = np.clip(sigmoid(X @ w), 1e-12, 1-1e-12)
    return -(y*np.log(p)+(1-y)*np.log(1-p)).sum()

def finite_difference(e):
    high = loss(X, y, w + 1e-6*e)
    low = loss(X, y, w - 1e-6*e)
    return (high - low) / 2e-6

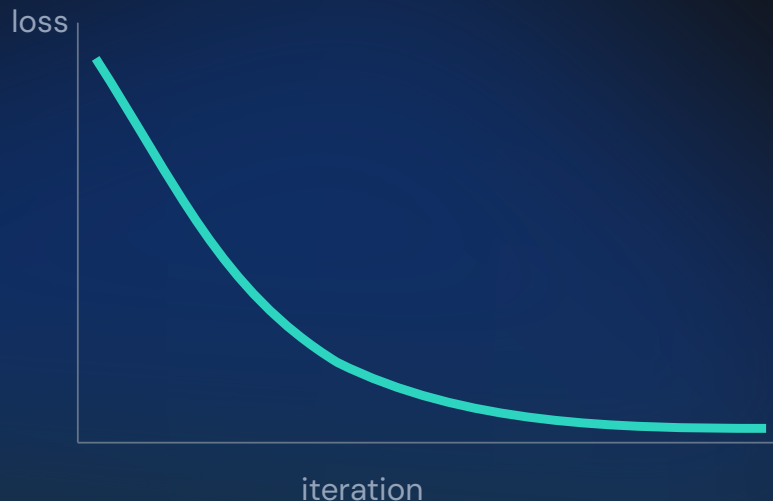
numeric = np.array([
    finite_difference(e) for e in np.eye(3)
])
analytical = gradient(X, y, w)
assert np.allclose(analytical, numeric)
```

Gradient Descent Is the Solution

$$w_{t+1} = w_t - \alpha X^\top (\hat{p}_t - y), \quad \hat{p}_t = \sigma(Xw_t)$$

```
def fit_logistic(X, y, alpha=.1, steps=1000):  
    Xb = np.c_[np.ones(len(X)), X]  
    w = np.zeros(Xb.shape[1])  
    for _ in range(steps):  
        p = sigmoid(Xb @ w)  
        w -= alpha * Xb.T @ (p - y) / len(y)  
    return w
```

```
w_fit = fit_logistic(X, y)  
assert np.isfinite(w_fit).all()
```



No closed form: $\hat{p} = \sigma(Xw)$ makes $X^\top(\hat{p}-y)=0$ nonlinear in w .

Regularized Logistic Regression

$$J_{L2}(w) = J(w) + \lambda \|w\|_2^2$$

- L2 or L1 controls overfitting exactly as in regression
- scikit-learn uses $C = 1/\lambda$
- **Smaller C means stronger regularization**
- Same bias-variance tradeoff, new loss

```
for C in [100., 1., .01]:  
    model = LogisticRegression(C=C, max_iter=1000)  
    model.fit(X, y)  
    size = np.abs(model.coef_).max()  
    print(f"C={C:6.2f}  max|w|={size:.3f}")
```

C=100
weak penalty
larger weights

C=1
balanced

C=.01
strong penalty
simpler fit

Beyond Two Classes: Softmax

$$P(y = k | x) = \frac{e^{x^\top w_k}}{\sum_{j=1}^K e^{x^\top w_j}}$$

$$J(W) = - \sum_i \sum_k 1[y_i = k] \log P(y_i = k | x_i)$$

scores

-1.2 · 0.7 · 2.1

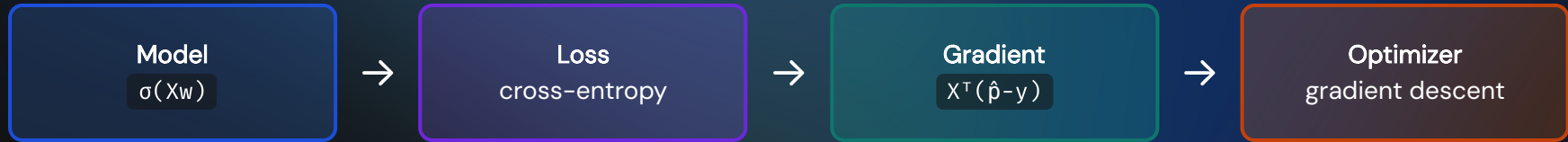
probabilities

.03 · .19 · .78

sum = 1

```
X3, y3 = make_classification(  
    n_samples=300, n_features=4,  
    n_classes=3, n_informative=3,  
    n_redundant=0, random_state=0)  
  
model = LogisticRegression(max_iter=1000)  
model.fit(X3, y3)  
probs = model.predict_proba(X3[:3])  
assert np.allclose(probs.sum(axis=1), 1)
```

Model → Loss → Gradient → Optimizer



Regularization controls complexity; softmax handles multiple classes.

Next: k-NN — classification with no loss, optimizer, or learned parameters.